

Orchestrating Intelligence: The Multi-Agent AI Revolution

Mike Erlihson, PhD, Head of AI, Drivenets



Agenda



From LLMs to Agents

Understanding the evolution and key differences

Multi-Agent AI Systems (MAAIS)

Why we need them and core components

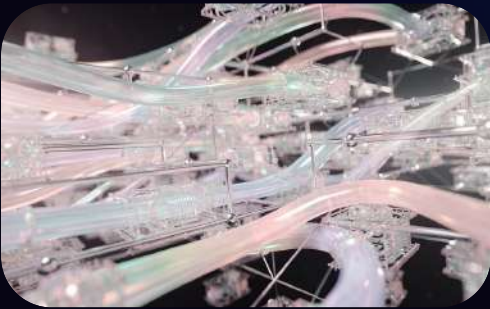
Orchestration Architectures

Centralized, Decentralized, Hierarchical, and Hybrid patterns

Agent Adaptation Paradigms

How agents and tools learn and improve to make MAAIS better

From LLMs to Multi-Agent AI Systems(MAAIS)



An LLM



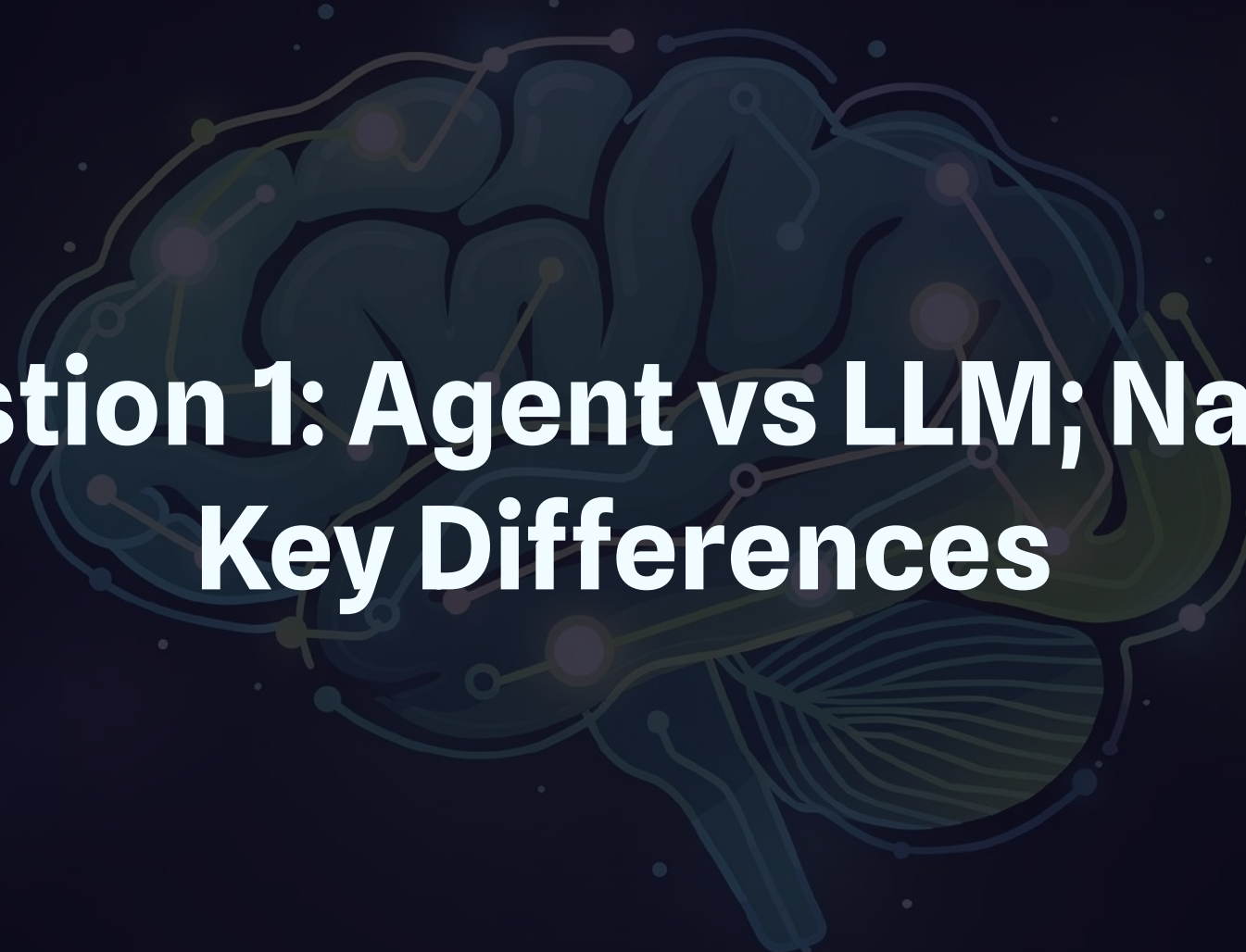
A Chatbot



An Agent



A Multi Agent AI System (MAAIS)



Question 1: Agent vs LLM; Name 5 Key Differences

Agent vs LLM, Difference 1



Core Identity

LLM: A statistical model that predicts the next best word; it is a knowledge engine.

Agent: A SW system using an LLM to make decisions; it is an execution engine.

Agent vs LLM, Difference 2



Operational Mode

LLM is Reactive: Waits for a human prompt & provides a single response before stopping.

Agent is Proactive: Operates in a loop (Plan→Act→Observe) until a goal is achieved.

Agent vs LLM, Difference 3



Tools & Capability

LLM: Limited to the data it was trained on; describes how to do things but can't do them.

Agent: Uses external **tools** (browsers, calculators, DBs, APIs) to take real-world action.

Agent vs LLM, Difference 4



Memory Depth

LLM: Has a "context window" (short-term memory) resetting once a conversation ends.

Agent: Often utilizes persistent memory to remember past mistakes & user preferences across different sessions.

Agent vs LLM, Difference 5



Decision Making

LLM: Focuses on linguistic patterns & following the immediate instruction.

Agent: Focuses on goal-oriented reasoning, breaking down a large task into smaller sub-tasks.

What is an AI Agent?

A computer program perceiving its environment, taking autonomous actions to achieve its goals



Operates independently

Makes informed (based on the env & its current state) decisions to accomplish its tasks

Utilizes LLMs(brain) for strategic planning & decision-making.

Component 1: Role/Persona

Agent's core identity & persona .

How it understands, responds, defines communication style, expertise & problem-solving patterns.

General

"You are a helpful
AI assistant."

Expert

"You are a senior
business analyst."

Specialized

"You are an expert
RUST developer."



Components 2-4: Brain, Memory & Planning Algo



The Brain

A decision-making core, powered by an LLM



The Memory

Stores past experiences, current context & important long-term knowledge.



Planning/Management Algo

Breaks down big user goals into smaller, easy-to-manage steps.

Learns from its own process to make plans better & fix mistakes early.

Component 5: Tools (Actions): "Outside world" Interaction

General Tools

- `web_search()`
- `file_read()`
- `file_write()`
- `http_request()`

Data Tools

- `sql_query()`
- `redis_get()`
- `elasticsearch_search()`
- `csv_analyze()`

Developer Tools

- `git_commit()`
- `pytest_run()`
- `docker_build()`
- `kubectl_apply()`



How Agents build Current Context?

Context is Key

Agents leverage their **Memory** & **Tools** to synthesize a dynamic understanding of the environment, enabling intelligent LLM decisions.

The Imperative for MAAIS

5 limitations of single AI agents

Why MAAIS is essential for complex, real-world challenges?



Limitation 1: Cognitive Overload (An LLM Context Window Curse, 1)

Single AI agents struggle with complex tasks.

Their ability to process and retain info quickly diminishes, leading to performance degradation.

A customer service AI handling multiple tasks **simultaneously** (order tracking/recommendations/refunds/tech support) provides generic responses instead of specialized solutions.

Limitation 2: Role Dilution(An LLM Context Window Curse, 2)

Attempting to be a "jack-of-all-trades" results in generic output.

Single agents lack the **deep, narrow expertise** & **specific tool-focus** of specialized subagents, leading to compromised quality.

An agent asked to write **both** a legal brief & marketing copy, producing technically correct text lacking the nuanced tone & specific terminology required for each domain.

Limitation 3: No Self-Correction Ability

Single AI agents lack an independent "critic" mechanism to verify their outputs, making them prone to uncorrected errors and hallucinations.

Without an internal **peer-review** process, agents struggle to identify & fix mistakes, leading to unreliable results.

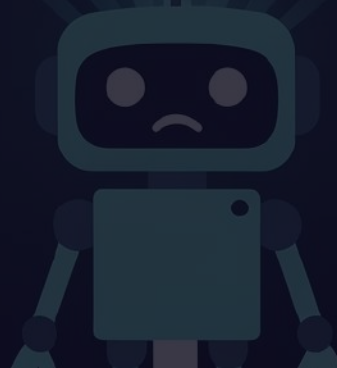
Example: An AI agent generating a **complex financial report** might **confidently** present **incorrect data/flawed conclusions** if it cannot independently cross-reference sources or validate its own reasoning.

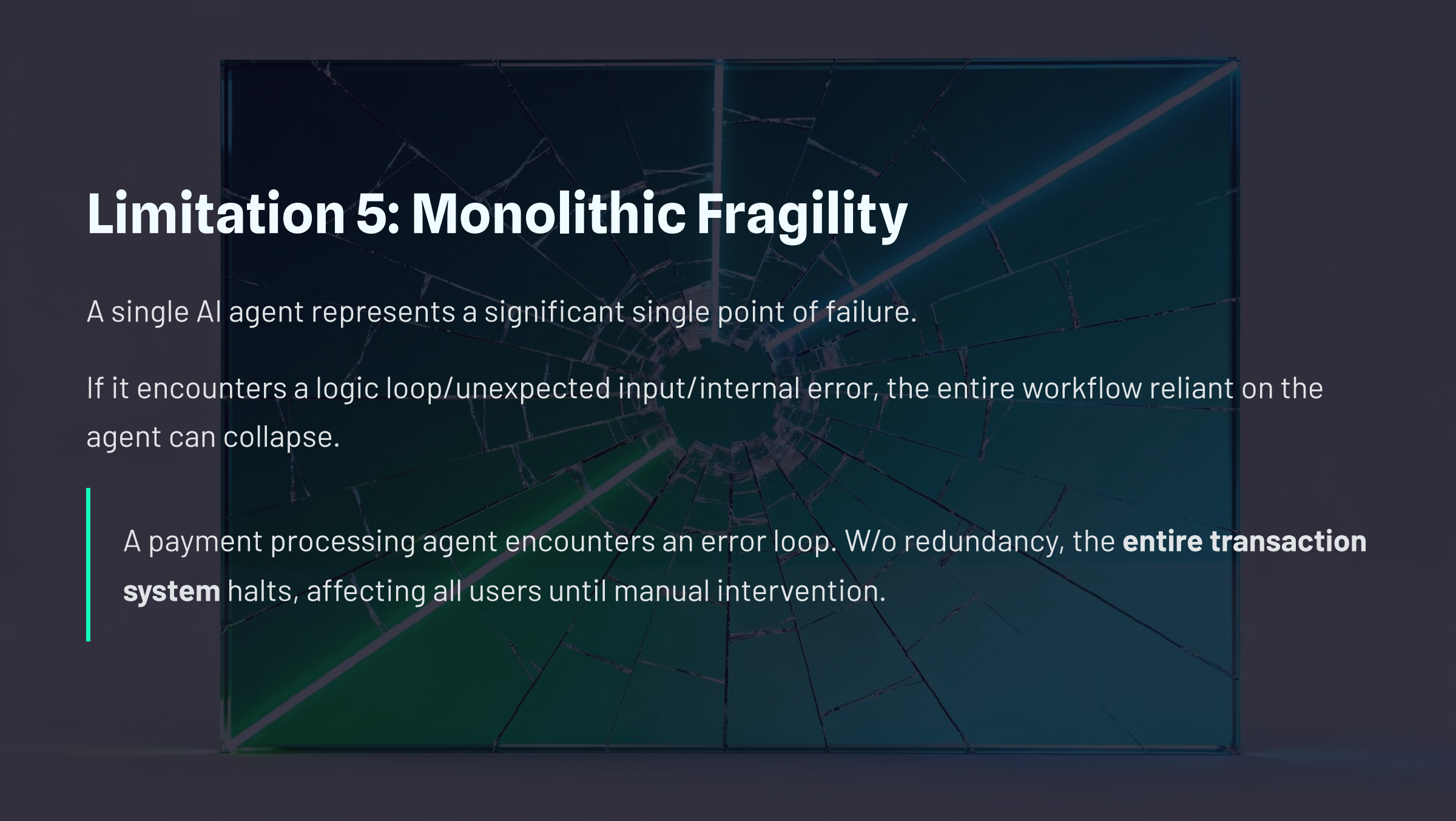
Limitation 4: Sequential (Slow) Latency

Single AI agents process tasks linearly, hindering overall efficiency.

Creates significant performance bottlenecks compared to the concurrent execution capabilities of MAAS.

An agent trying to perform simultaneous tasks like compiling code, running tests & deploying updates will execute each step one after another, slowing down the entire SW development lifecycle.



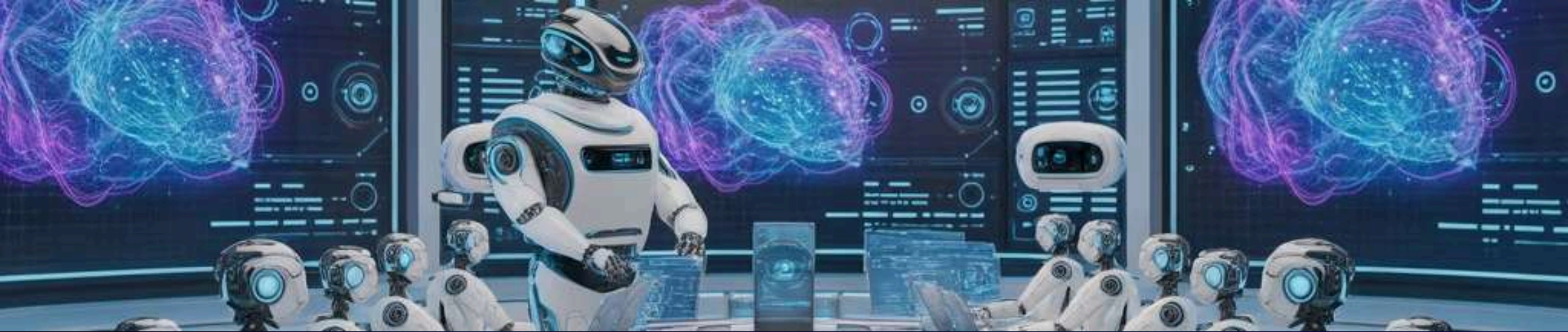


Limitation 5: Monolithic Fragility

A single AI agent represents a significant single point of failure.

If it encounters a logic loop/unexpected input/internal error, the entire workflow reliant on the agent can collapse.

A payment processing agent encounters an error loop. W/o redundancy, the **entire transaction system** halts, affecting all users until manual intervention.



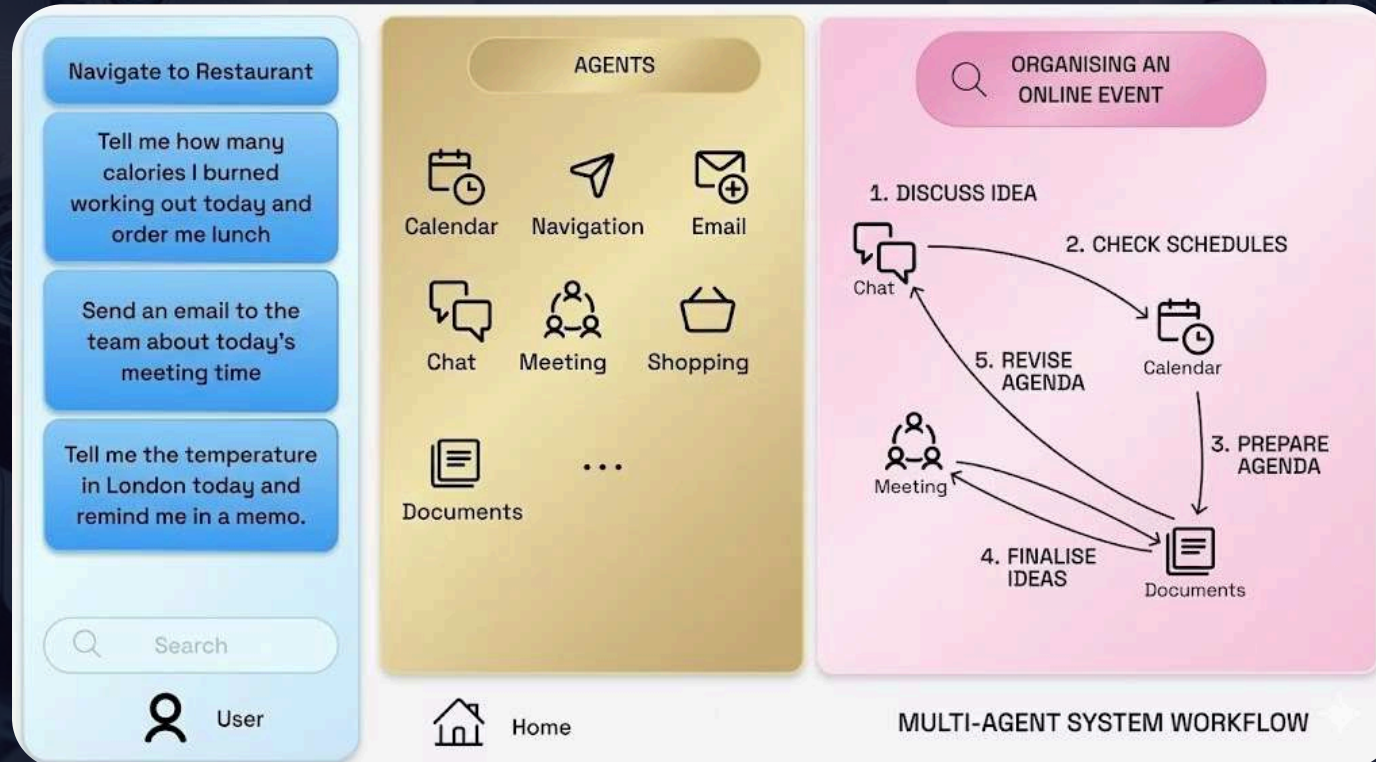
From an AI Agent —> MAAIS

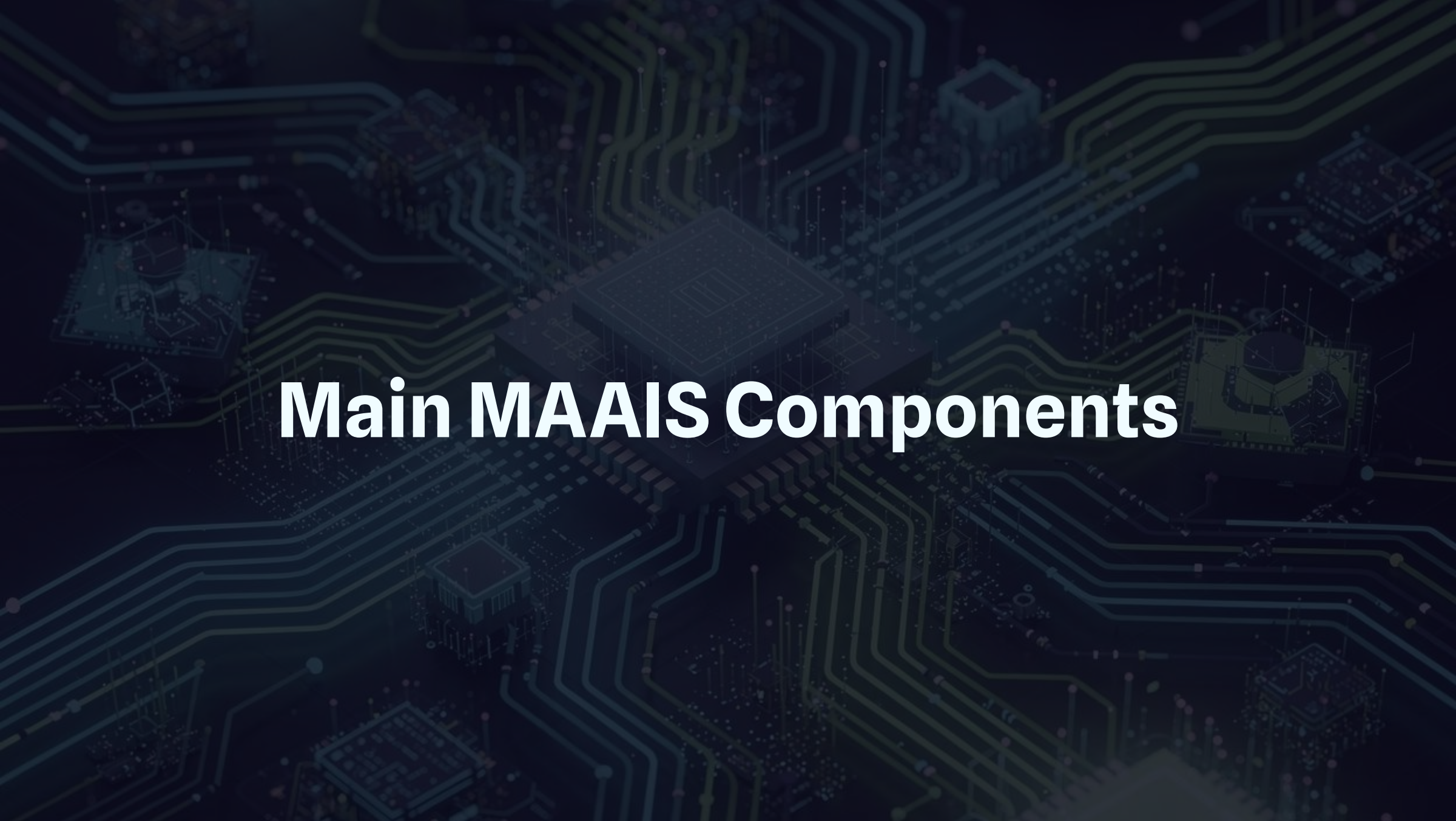
MAAIS a collection of autonomous, specialized AI agents that collaborate, coordinate, or compete in a shared environment to solve complex problems, leveraging individual capabilities & distributed intelligence for tasks too difficult for a single AI agent.

Many Coordinated Agents

Instead of a single AI model(agent), a coordinated team of **specialized** agents works together.

Each agent excels at specific tasks, delivering superior accuracy and efficiency.





Main MAAIS Components



Agents: The Specialists

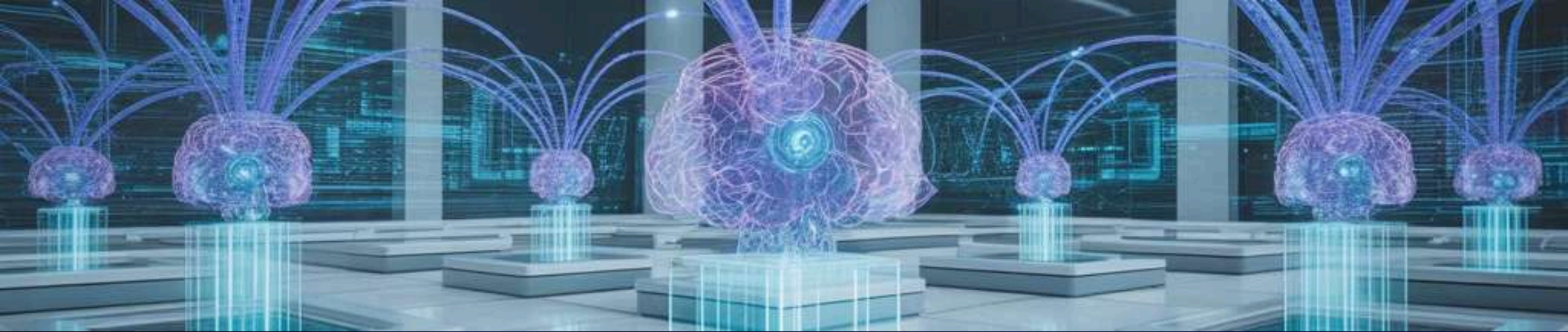
Specialized agents with its roles, rules & skills.

SW development: Coder, Reviewer & Manager agents.

Orchestration

Defines how specialized agents execute tasks together

Types: From sequential execution (Agent $A \rightarrow B \rightarrow C$) to a hierarchical 'Boss $\rightarrow$ agents' operations



Agent Communication

Facilitates info exchange among agents

Methods: NL or JSON, YAML.

Shared State: The Collective Memory

A central workspace where all agents can view and modify shared info.

When to use MAAIS the right fit?

Dynamic Collaboration is Essential

For scenarios requiring continuous, adaptive problem-solving and dynamic decision-making among specialized agents.

A Single Agent Is Insufficient

MAAIS distributes the workload with specialized agents & enhances reliability through redundancy.

The Universal Interface: Everything as a Tool

Breaking the Hierarchy in Multi-Agent Systems

1

Old Way of Thinking

Tools are simple, separate functions that couldn't easily work together.

2

Universal Interface

A "tool" is any function that can be reached & used in a standard way. This changes how we define tools.

3

Layered Systems

Agents can use other agents or their own parts as tools. This creates deep, complex systems, like an **"Inception" dream**.



INCEPTION

The Universal Interface: Everything as a Tool

Sub-Agent	A subordinate to manage.	An LLM-backed endpoint that consumes a prompt & returns a structured result, identical in interface to a calculator.
Memory	DB/KV store/context window.	A Read/Write interface. An agent calls a tool to store a fact/calls a retrieve function to recall one.
External Function	An API call (e.g., Google Search).	A bridge to the physical/digital world (Python, APIs, SQL) providing data the LLM cannot generate.



MAAIS orchestration architectures

Orchestration Type 1: Centralized

A single agent coordinates all tasks, data flow (routing/decision making) for other agents

Pros

- Clear control
- Simplified management
- Consistency in decisions

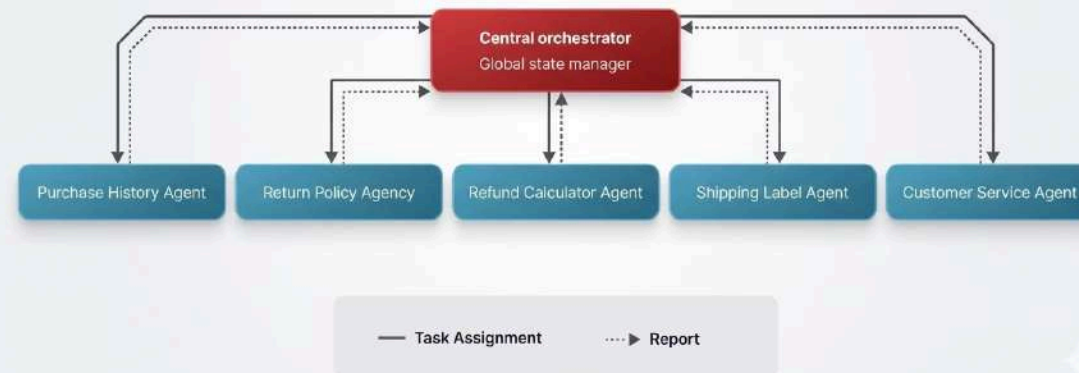
Cons

- Potential single point of failure (bottlenecks)
- Less adaptable to rapidly changing systems

A customer service Agent

Centralized orchestrator architecture

Like a conductor leading an orchestra – Single point of control



Centralized Architecture: An Anthropic's Research Agent Example

1

Lead Agent Initializes

Analyzes complex query like "**emerging AI startups in healthcare**," then develops a strategic plan.

2

Subagent: Funding Data

Spawns a specialized agent to explore funding landscapes and investment trends.

3

Subagent: Clinical Applications

Deploys another agent to investigate medical & clinical use cases.

4

Subagent: Regulatory Landscape

Activates an agent focusing on legal & compliance aspects of healthcare AI.

5

Subagent: Competitive Dynamics

Assigns an agent to analyze market competition & key players.

6

Orchestrator Synthesizes

Collects findings from all subagents & compiles them into a coherent, comprehensive response.

Centralized Architecture: Performance Characteristics

High Token Efficiency

With no redundant processing, token usage remains efficient as the central agent directs tasks precisely.

Increased Latency

Sequential coordination by the orchestrator introduces overhead, leading to longer response times for complex operations.

Throughput Ceiling

The system's overall processing capacity is limited by the single orchestrator, creating a potential bottleneck at scale.

Concentrated Context

All critical operational context & decision-making logic reside within the central agent, simplifying state management.

Orchestration Type 2: Decentralized

Each agent operates autonomously as a peer, sharing info & coordinating directly with others without a central boss.

Pros

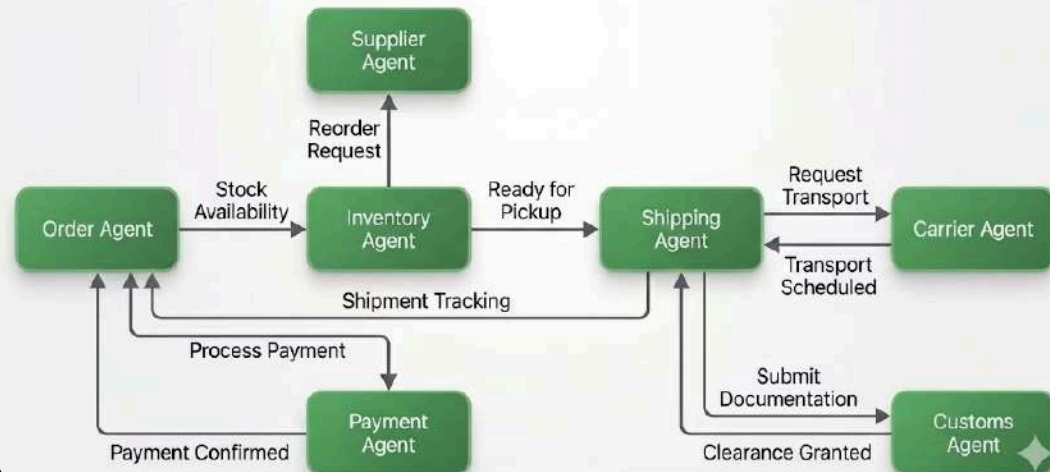
- High flexibility
- Highly adaptable to dynamic environments
- No single point of failure

Cons

- Requires sophisticated communication protocols
- Higher complexity to debug and maintain

Decentralized Supply Chain Logistics Workflow

Agents negotiate directly – No central authority



Decentralized Orchestration: Supply Chain Logistics System

Order & Inventory: The **Order Agent** queries the **Inventory Agent** for stock. If low, the **Inventory Agent** triggers a reorder with the **Supplier Agent**.

Shipping & Carrier: The **Inventory Agent** notifies the **Shipping Agent** when items are ready. The **Shipping Agent** then coordinates transport with the **Carrier Agent**.

Customs & Payment: For international shipments, the **Shipping Agent** coordinates with the **Customs Agent**. Simultaneously, the **Order Agent** processes payment with the **Payment Agent**.

Final Coordination: Once transport is scheduled & customs cleared, the **Shipping Agent** provides tracking details to the **Order Agent**, completing the fulfillment loop.

Decentralized Orchestration: Performance Characteristics

Reduced Token Efficiency

Decentralized agents can lead to redundant tasks and queries, increasing overall token usage.

Lower Local Latency

Independent agent decisions reduce central communication overhead, enabling faster local task completion.

Scalable Throughput

Capacity scales linearly with more agents, distributing workloads & preventing bottlenecks.

Distributed Context

Operational context & decision logic are distributed, enhancing resilience & reducing single points of failure.

Orchestration Type 3: Hierarchical

Higher-level agents supervise & strategize for lower-level agents, who execute specific tasks.

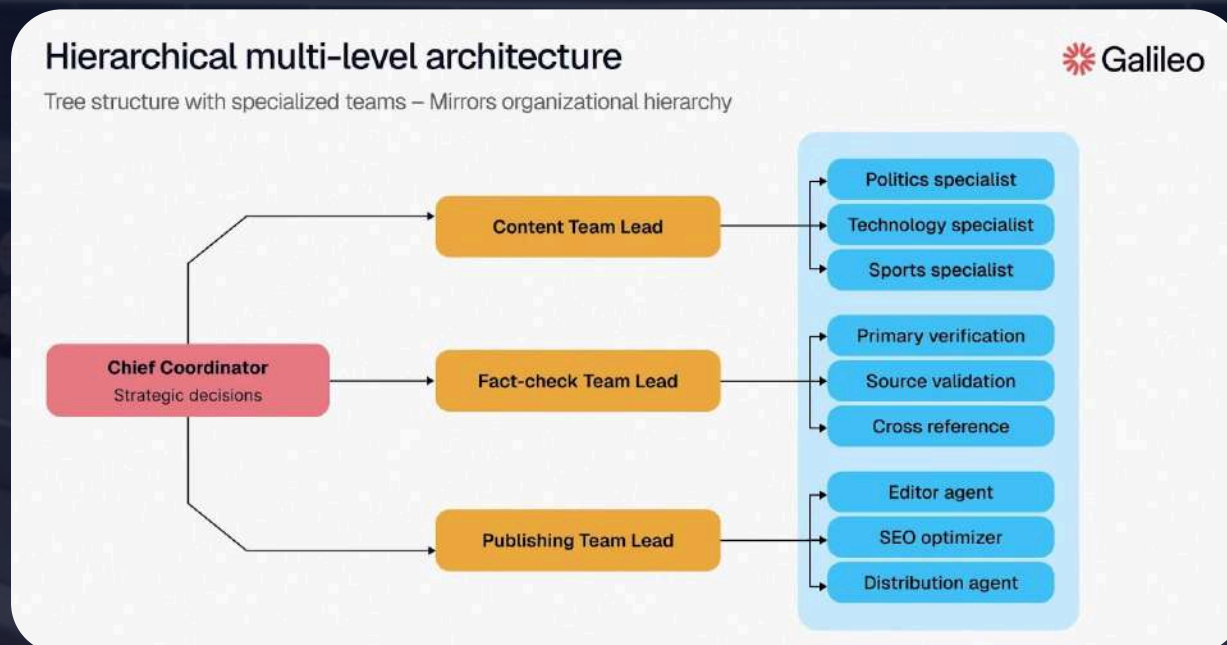
Pros

- Balances flexibility and oversight
- Ideal for scaling complex systems by breaking down goals

Cons

- Coordination across layers can be complex
- Potential latency due to dependency delays

News aggregation platform



Hierarchical Orchestration: News Aggregation Platform



Top Supervisor Agent

Oversees everything: gathering news, checking facts, and deciding what to publish.



Content Supervisors

Manages teams of agents specializing in different topics like Politics, Tech, or Sports.



News Gatherers

Their job is to find and collect raw news from many different sources for their specific topics.



Fact-Checking Manager

Manages agents who check facts using different ways to make sure all news is accurate & trustworthy.



Publishing Team Lead

Coordinates agents who format, schedule, and distribute approved articles across different platforms and channels.

Hierarchical Orchestration: Performance Characteristics



Token Efficiency

Token efficiency is moderate with some redundancy between levels



Latency

Latency is moderate due to multi-hop coordination



Throughput

Throughput is high through parallel teams



Context Segmentation

Context segments by level and team

Orchestration Type 4: Hybrid(Federated)

Multiple independent agent systems collaborate across different organizations/domains using shared protocols.

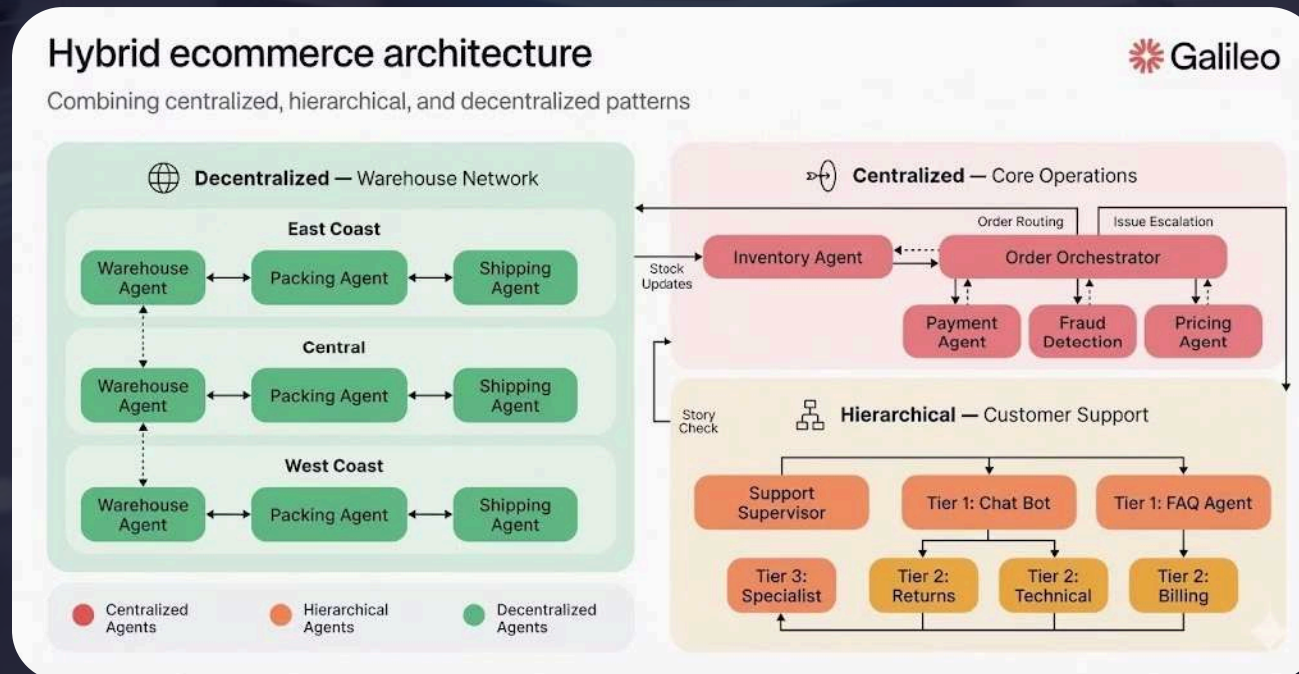
Pros

- Facilitates cross-system collaboration
- Leverages specific strengths of different systems

Cons

- Relies heavily on interoperability & shared standards
- Security and trust management can be difficult

Ecommerce System Agent



Hybrid Orchestration: Ecommerce System

Hybrid Approach:

Mixes 3 different styles to handle specific business needs efficiently.

Decentralized (Warehouses):

Regional warehouses operate independently to ship orders faster and avoid bottlenecks.

Centralized (Core Ops):

A single "brain" controls the big decisions like inventory, payments, and security.

Hierarchical (Support):

Support is organized in levels, bots handle simple questions, passing harder ones up to human experts.

Connected System:

All parts share data instantly, ensuring the local teams and the main office stay in sync.

Hybrid Orchestration: Performance Characteristics



Adaptability

Performance adapts to requirements



Token Efficiency

Varies based on task distribution



Latency

Optimizes for both global and local operations



Throughput

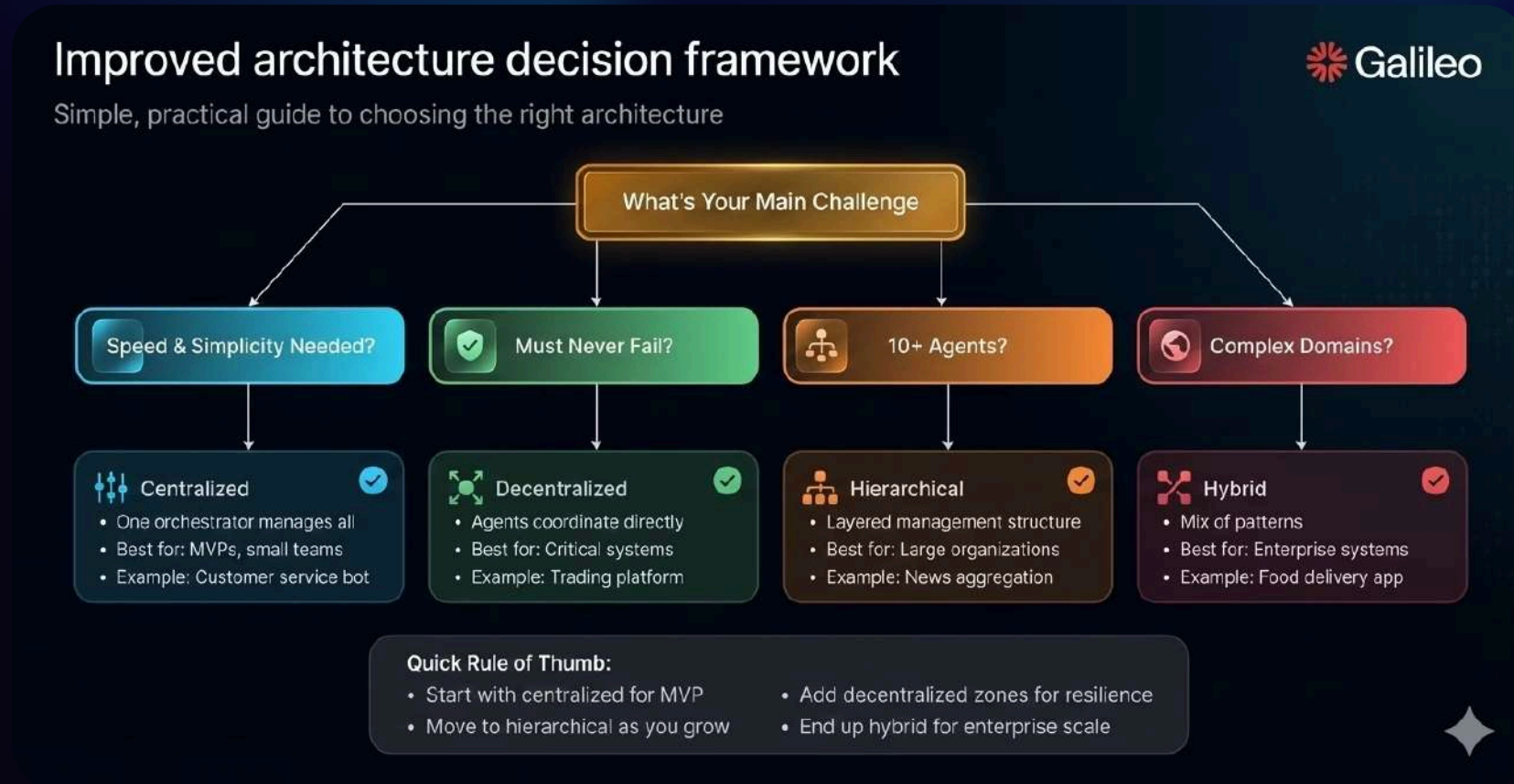
Combines the benefits of both approaches



Strategy

Context is strategic at center, tactical at edges

How to choose orchestration for your task?





**Question 2: Can you think of another
taxonomy of MAAIS?**

Static vs. Dynamic MAAIS

A Static Graph

Rely on **pre-programmed logic**. Humans define the interaction graph & state transitions before deployment, meaning agents follow a rigid, pre-determined path(flow).

A Dynamic Graph

An orchestrator uses real-time intelligence to select agents, tools, sequences, adapting dynamically to the evolving task context

MAAIS potential is huge but key challenges remain

Show remarkable promise in domains from scientific discovery to SW development.

However, still struggle with critical limitations that prevent widespread, reliable deployment.

MAAIS Shortcomings: Key Challenges

Unreliable Tool Use

Agents frequently misuse/misunderstand tools, leading to incorrect actions or failed operations.

Limited Long-Horizon Planning

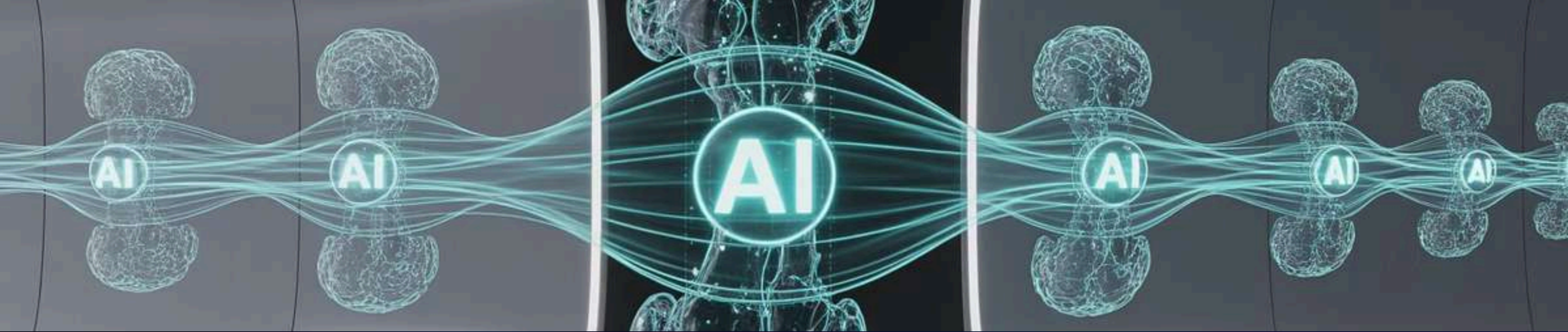
Performance degrades significantly over multi-step tasks requiring sustained reasoning and coordination.

Domain-Specific Reasoning Gaps

Generalist LLMs lack the specialized knowledge needed for expert-level performance in specific domains.

Poor Generalization

Agents struggle in unexplored environments where they lack prior experience or training data.



Adaptation: The Engine of MAAIS Excellence

A critical mechanism to overcome limitations and unlock specialized, high-performance MAAIS

Two Axes of Adaptation

1. What is being optimized?

2. Where the feedback signal originates?



Agent Adaptation

Optimize internal model parameters and behavior.



Tool Adaptation

Tune external components, APIs, and tools.

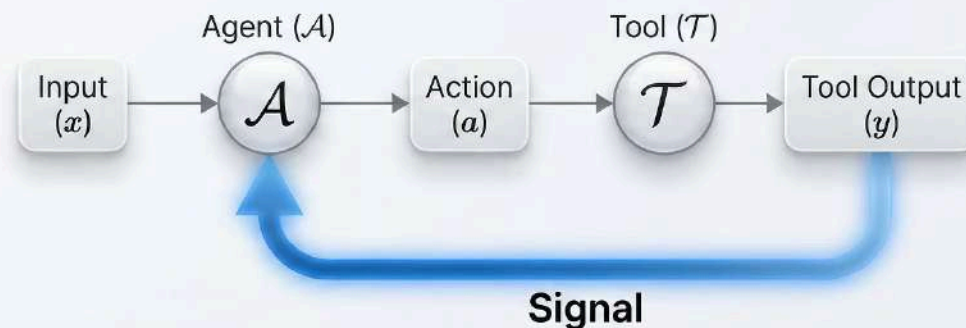
4 Paradigms of Agentic AI Adaptation



Paradigm A1: Tool Execution Signaled Model Adaptation

Core Definition

The agent ($\mathcal{A}$) is optimized using verifiable outcomes produced by the external tools ($\mathcal{T}$) it invokes. The signal is objective and grounded in the tool's execution.



Formal Optimization Objective

$$A^* = \operatorname{argmax}_A \mathcal{O}_{tool}(A, T)$$

$\mathcal{O}_{tool}$ measures the quality of the tool's outcome, such as API success rates, retrieval scores, or code execution pass rates.

A1 in Practice: Imitation vs. Interaction

Supervised Fine-Tuning (SFT) & Off-Policy

The agent learns to imitate successful tool-using behavior from a pre-collected dataset $\mathcal{D}_{\text{succ}}$ of $(\mathbf{x}, a^*)$.

Objective

$$A^* = \operatorname{argmax}_A \mathbb{E}_{(x, a^*) \sim \mathcal{D}_{\text{succ}}} [\log p_A(a^* | \mathbf{x})]$$

Seminal Examples

- Toolformer, ToolLLM, Gorilla.

Reinforcement Learning (RL) / RLVR

The agent learns through online interaction, receiving a reward R based on the tool's output y .

Objective

$$J(A) = \mathbb{E}_{x, a, y} [\mathcal{O}_{\text{tool}}(\mathbf{y})]$$

where the reward is defined as $R = \mathcal{O}_{\text{tool}}(\mathbf{y})$.

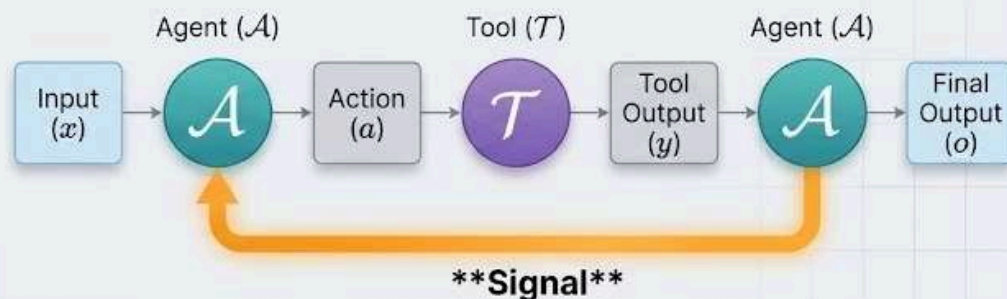
Seminal Examples

- DeepRetrieval, DeepSeek-Prover-V2, Code-R1.

Paradigm A2: Agent Output Signal Agent Adaptation

Core Definition

The agent ($\mathcal{A}$) is optimized using evaluations of its own final outputs (o), such as answers, plans, or reasoning traces.



Formal Optimization Objective

$$A^* = \operatorname{argmax}_A \mathcal{O}_{agent}(A, T)$$

$\mathcal{O}_{agent}$ evaluates the quality of the agent's final generated output o , such as correctness, quality, or alignment.

Paradigm A2: With/Without Tool Adaptation

1

Agent Adaptation w/o Tools:

Improves intrinsic reasoning abilities (e.g., math, coding) by evaluating generated solutions, without involving external tools.

2

Agent Adaptation w/ Tools:

Adapts by assessing outputs and tool interactions. Feedback enhances tool planning, selection, and execution based on tool-mediated task outcomes.

A2 in Practice: Dual Supervision vs. Output Driven RL

Supervised Fine-Tuning (SFT)

Concept: To learn tool use, SFT must combine A1-style imitation of the tool call (a^*) with A2-style imitation of the final answer (a^*).

Objective:

$$A^* = \operatorname{argmax}_A \mathbb{E} [\log p_A(a^*|x) + \log p_A(o^*|x, a^*, y)]$$

Seminal Examples (w/ Tools):

Self-RAG, FireAct.

Reinforcement Learning (RL)

Concept: The agent receives a reward R based solely on the quality of its final output o , regardless of intermediate tool use.

Objective:

$$J(A) = \mathbb{E}_{x,a,y,o} [O_{\text{agent}}(o)]$$
$$R = O_{\text{agent}}(o)$$

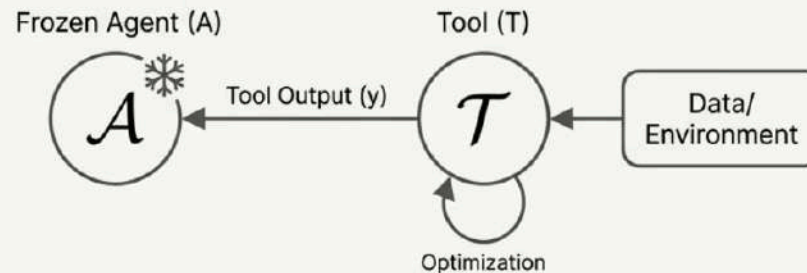
Seminal Examples:

- **w/o Tools:** DeepSeek-R1 (Math), Kimi-1.5.
- **w/ Tools:** Search-R1, ReTool.

Paradigm T1: Agent-Agnostic Tool Adaptation

Core Definition

Tools ($\mathcal{T}$) are trained independently of the frozen agent ($\mathcal{A}$). This is standard model training, creating plug-and-play modules.



Formal Optimization Objective

$$T^* = \operatorname{argmax}_T \mathcal{O}_{\text{tool}}(T)$$

$\mathcal{O}_{\text{tool}}(T)$ evaluates the quality of the tool-produced results directly (e.g., retrieval accuracy, simulation fidelity), independent of any specific agent.

T1: A Universe of Pretrained Capabilities

Foundational Systems

Architectures like HuggingGPT and ViperGPT that orchestrate T1 tools.

Vision Models

CLIP, Segment Anything Model (SAM).



Speech & Audio

Whisper for transcription and translation.



Search & Retrieval

Dense retrievers like DPR and ColBERT.



Scientific Tools

AlphaFold2 (protein folding), CGCNN (materials science).



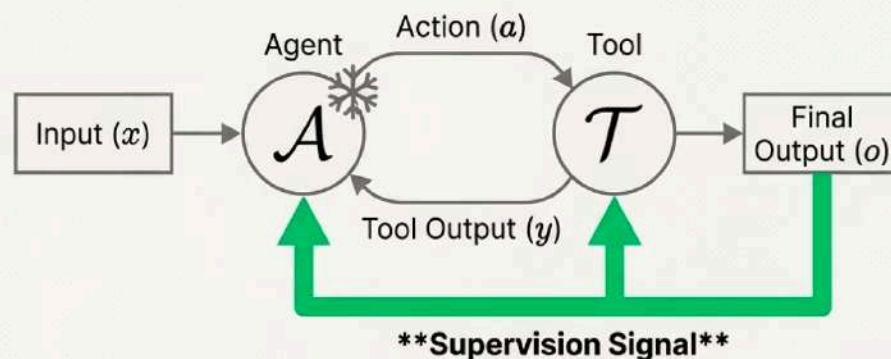
Subagent-as-Tool

An agent trained via A1 (e.g., DeepRetrieval) can be frozen and used as a specialized T1 tool by another agent.

Paradigm T2: Agent-Supervised Tool Adaptation

Core Definition

The agent (A , as in $\mathcal{A}$) remains fixed, while its tools (T , as in $\mathcal{T}$) are adapted using signals derived from the agent's own outputs (o).



Formal Optimization Objective

$$T^* = \operatorname{argmax}_T \mathcal{O}_{agent}(A, T)$$

The tool ' T ' is optimized to improve the performance of the *fixed* agent-tool system, as measured by the agent's final output.

T2 in Practice: Agent-Driven Learning & Adaptive Memory

Agent-Supervised Learning

- The tool learns from the agent's final output, either via a quality score $w(o)$ (Quality-Weighted Training) or an implicit target τ (Output-Consistency Training).

Seminal Examples: S3, AgentFlow.

Reinforcement Learning (RL)

- The tool receives a reward R based on the agent's final output o , and is updated via RL.

Objective:

$$J(T) = \mathbb{E}_{x,a,y,o} [O_{\text{agent}}(o)]$$

Agentic Memory as a Special Case of T2

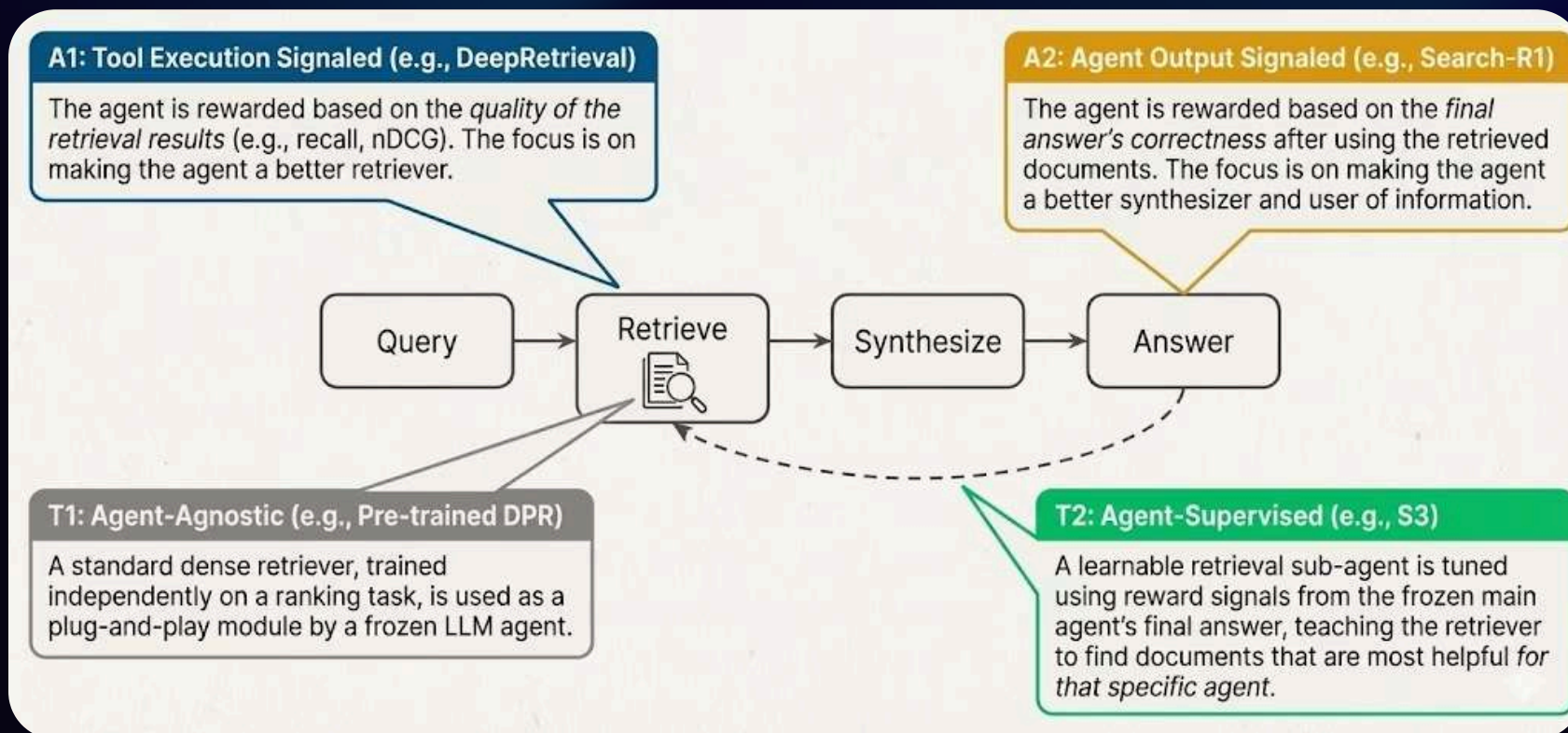
The memory module (**T**) is updated based on the agent's (**A**) final output (**o**) via a write function: $M \leftarrow \text{Update}(M, o)$. The memory adapts to better support the frozen agent's future reasoning.

Examples: Reflexion, Memento.

THE GRAND COMPARISON: A STRATEGIC GUIDE TO ADAPTATION

	A1 Paradigm (Agent Adaptation)	A2 Paradigm (Output-Driven Agent Adaptation)	T1 Paradigm (Tool Adaptation)	T2 Paradigm (Agent-Supervised Tool Adaptation)
Cost & Flexibility	High computational cost (training large models), but maximal flexibility in adapting agent behavior.		Lower cost (optimizing smaller tools), but constrained by the frozen agent's capabilities.	
Generalization	May overfit to specific tool environments unless carefully regularized.	Can generalize reasoning but depends on the quality of output signals.	Often generalize well across agents and tasks if trained on broad data.	Specialized to the supervising agent; may not transfer well.
Modularity	Low. Adapting the agent for new tasks risks catastrophic forgetting.		High. Tools can be upgraded or swapped independently without retraining the agent, facilitating continuous improvement. ✨	

4 Adaptation Paradigms in a Single Example



Agent Adaptation Techniques: Trends



Transition from imitation-based training (SFT & DPO with static datasets) to reinforcement learning approaches (RLVR with dynamic, online interactions)



No tool reasoning agent (model) training extended to tool-augmented settings, training agents to make better decisions about when & how to invoke tools based on final task success.

Tool Adaptation Techniques Trends



Early Methods: Proxy Signals

Simple tools (e.g. retrievers) were trained using indirect proxy signals from a frozen agent, such as output perplexity/basic internal preferences.



State-of-the-Art: Outcome-Driven RL

Complex tools (sub-agents/memory modules) are trained with outcome-driven RL, using rewards based on the frozen agent's final task success.

The Next Frontier: Open Challenges in Agentic Adaptation

Co-Adaptation

How can we efficiently & stably optimize agents & their tools jointly?

Continual Adaptation

How can systems adapt to new tasks & evolving tools in non-stationary environments without catastrophic forgetting?

Safe Adaptation

How do we ensure that adapted agents remain aligned, robust, & predictable, especially when learning online?

Efficient Adaptation

What are the most sample-and compute-efficient methods for adapting billion-parameter models & their complex tool ecosystems?

What We've Learned Today



AI Agents

Autonomous entities with roles, memory, planning, & tools to interact with their environment.



MAAIS Foundations

MAAIS leverage coordinated agents to tackle complex problems beyond single AI capabilities.



Orchestration Architectures(Patterns)

Various models (Centralized, Decentralized, Hierarchical, Hybrid) dictate how agents collaborate effectively.



Agentic Adaptation

Mechanisms for agents & tools to learn, evolve, & optimize performance through feedback & experience.

The Bonus Slide

A "Developer" Paradigm Shift: From Operator to Architect

The Traditional Developer: Operator

Manually crafts every line of code.

Implements intricate, low-level logic.

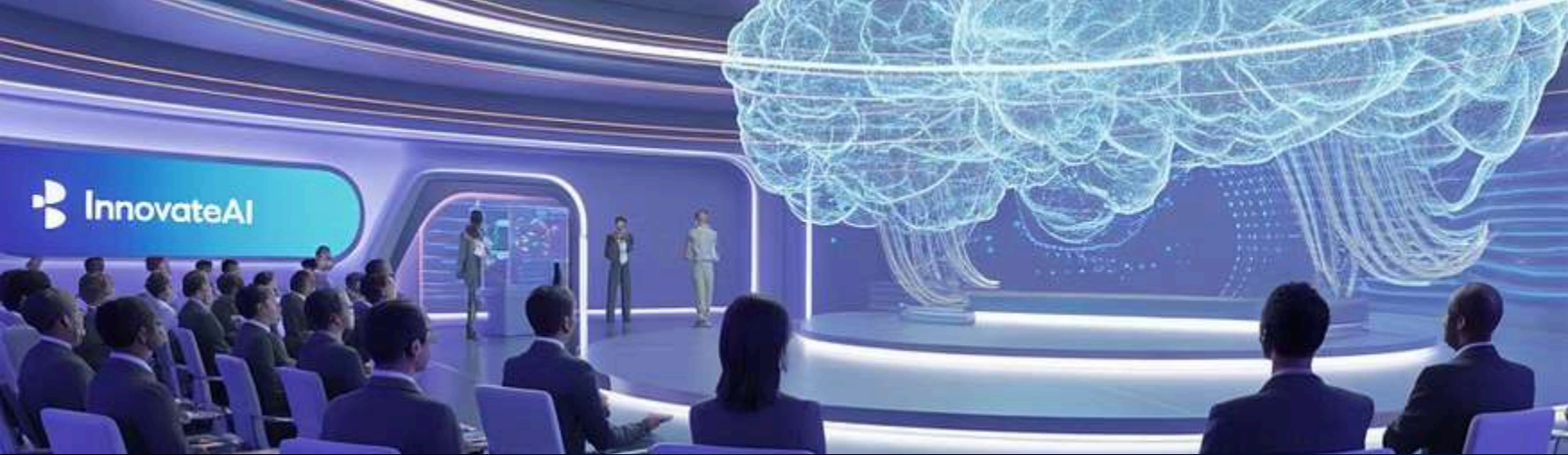
Directly interacts with system execution.

The Agent Architect: Designer

Orchestrates intelligent agent behavior.

Manages & integrates diverse toolchains.

Defines & optimizes high-level workflows.



**The future.
Intelligent.**

Questions, Answers(Hopefully) & Discussion

Framework Landscape



LangChain

Comprehensive framework with extensive tool integrations, memory management, and chain orchestration capabilities.



LangGraph

Framework for building stateful, multi-actor applications with LLMs, focusing on graph-based workflows and state management.



Crew AI

Framework for orchestrating role-playing autonomous AI agents for collaborative task execution.

Mission 3: Software Developer Agent

1

Role (System Prompt)

"You are an expert SW developer. Your mission is to autonomously implement, debug, and refactor code, ensuring high quality and adherence to best practices."

2

Tools & Data

```
file_system.read(path), file_system.write(path,  
content), run_tests(file_path),  
git_commit(message)
```

Access to codebase, documentation, test suite

3

Engine

Gemini 1.5 Pro

4

Current Instruction

"Implement a new feature: add user authentication via OAuth to the `auth.py` module. Ensure all existing tests pass."

More Real-World Agent Applications



Automated Debugging

Agents that can analyze stack traces, check logs, run diagnostics, and even apply fixes to common issues automatically.



Security Monitoring

Continuous agents that monitor for suspicious activity, analyze threats, and implement containment measures.



Code Review Automation

Agents that understand your codebase standards, run tests, check for vulnerabilities, and suggest improvements.



Data Pipeline Management

Agents that monitor data quality, handle failures, optimize queries, and maintain ETL processes.

Getting Started Today



Pick a Framework

Start with LangChain for general use cases, or Autogen for multi-agent scenarios. Both have excellent documentation.



Create Your First Agent

Combine your tool with a clear role and test it on real tasks from your daily workflow.



Build One Simple Tool

Create a tool for something you do regularly: file operations, API calls, or database queries.



Iterate and Expand

Add more tools, refine the role, experiment with different engines, and tackle more complex scenarios.

Building Your First Agent: Step by Step

Define the Role

Start with a clear, specific persona. What expertise does your agent need? What's its primary responsibility?

Build Essential Tools

Start with 2-3 core tools. Test each one individually before combining them in agent workflows.

Choose Your Engine

Select an LLM based on reasoning capability, cost, and speed requirements for your use case.

Test Simple Instructions

Begin with basic tasks to validate your agent setup before moving to complex multi-step scenarios.

Key Takeaways



Agent = Role + Brain + Memory + Tools

4 components: role, LLM for decisions, memory for context, and tools for actions.



ReAct: Think → Act → Learn

Continuous loop: reason about goals, execute actions, observe results, adapt.



Multi-Agent Systems (AAIS)

Specialized agents collaborate through orchestration and shared state using Sequential, Parallel, Hierarchical, or Peer-to-Peer patterns.

Key Takeaways: Implementation & Future

Paradigm Shift: Traditional2Agent Architecture

- Moves from explicit, step-by-step programming to dynamic, autonomous behavior
- Enables more complex problem-solving and adaptive, self-improving systems
- Focus shifts from "how to do" to "what to achieve"

Practical Implementation Considerations

- Scalability: Design for growth and concurrent operations
- Security: Implement robust access controls and data protection
- Testing & Debugging: Develop strategies for non-deterministic behavior
- Monitoring: Track agent performance and interaction logs for insights

Specialized Agent Examples



Data Fetcher

Specializes in retrieving information from diverse databases, APIs, and web sources.



Data Analyst

Focuses on processing, interpreting, and extracting insights from raw data.



Content Creator

Generates reports, summaries, or drafts based on analyzed information.



Action Executor

Handles the execution of decisions, interacting with external systems or tools.

Micro-Agent Philosophy

Specialized Responsibility

AAS benefit from a "micro-agent" philosophy: each agent has a narrow, well-defined responsibility.

Modularity & Reusability

Promotes modularity, reusability & easier debugging.

Microservices Principles

Mirrors principles from microservices architecture.

Expertise in Domain

Each agent becomes a specialized expert in one domain.

Mission 2: Execution Trace Agent

Thought(T): High CPU alert on user-service. I need to check current pod status & resource usage.

Action(A): `kubectl("get pods -l app=user-service -o wide")`

Observation(O): 3 pods running, all showing high CPU usage around 85-90%

T: Let me check the current resource limits and HPA status.

A: `kubectl("describe hpa user-service-hpa")`

O: HPA is at maximum replicas (3), target CPU is 70%, current is 88%

T: Need to scale up manually and create incident ticket.

A: `kubectl("scale deployment user-service --replicas=6")`

O: Deployment scaled successfully to 6 replicas

T: Let me generate a jira ticket this action

A: `jira_create_ticket(summary="HIGH: user-service CPU spike", description="Auto-scaled from 3 to 6 replicas due to 88% CPU usage", priority="High")`

O: Ticket PROD-4324 created

Final Answer: Incident resolved. Scaled user-service from 3 to 6 replicas. CPU usage normalized to 45%. Created ticket PROD-1234 for follow-up investigation.

Agents: Performance & Observability Challenges

Evaluating Agent Performance

Comprehensive evaluation requires measuring task completion, output quality, token efficiency, and latency across diverse scenarios.

Monitoring Complex Traces

Debugging multi-step ReAct loops and tool invocations is challenging; comprehensive tracing is essential to identify when agents go "off-rails."

Alerting on Anomalies

Real-time detection of agent failures, hallucinations, or tool misuse is critical, requiring baselines and effective alerts.

Mission 2: On-Call Engineer Agent

1

Role (System Prompt)

"You are an expert Site Reliability Engineer. Your mission is to ensure system stability by diagnosing & mitigating production alerts with speed & precision."

2

Tools & Data

`kubectl(command)`

`jira_create_ticket(...)`

`prometheus_query(...)`

3

Current Instruction

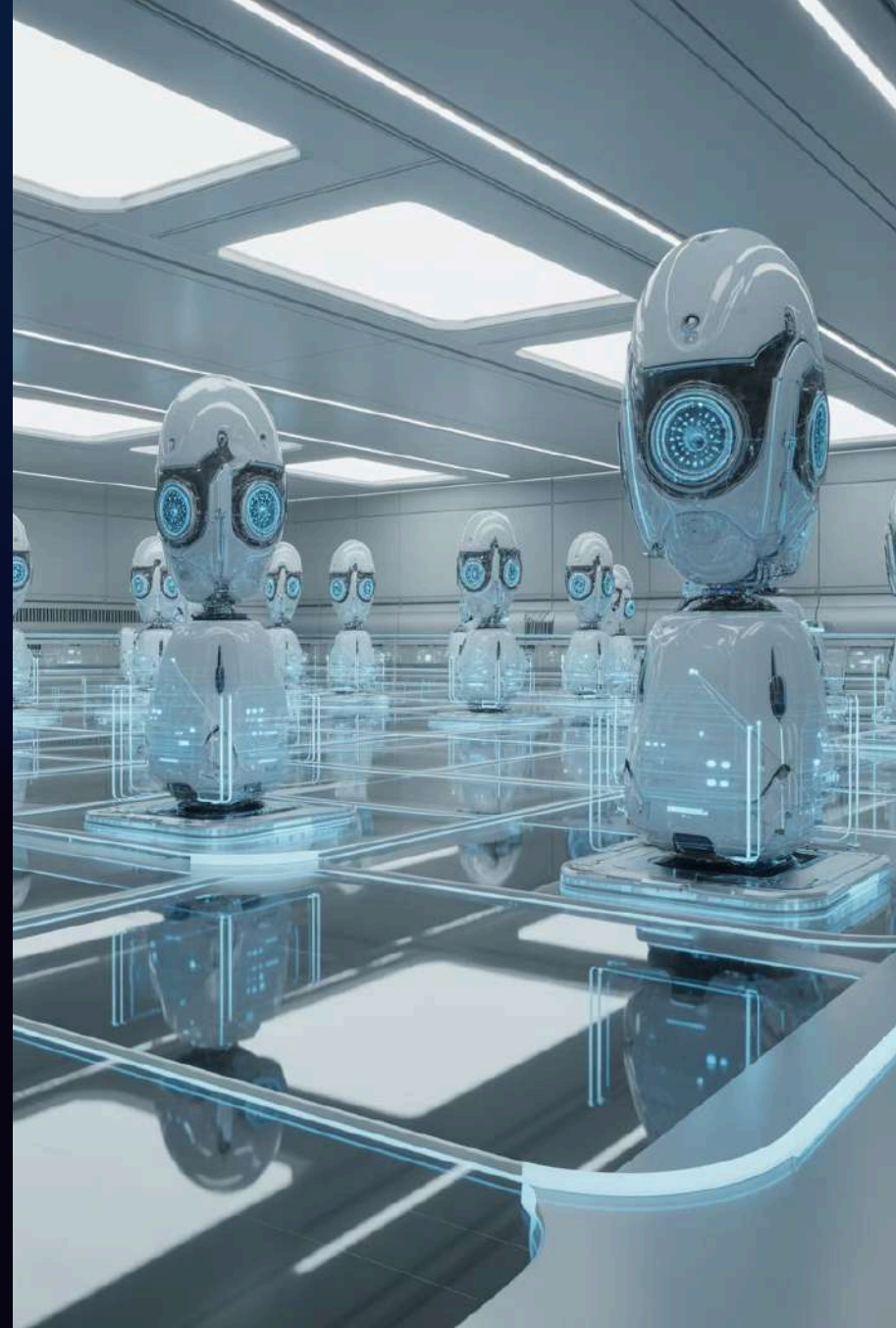
"Prometheus alert received: High CPU usage on 'user-service' deployment. Investigate and remediate."

Dynamic Context(Flow) in Action

Customer Interaction Flow

- Customer asks: "Can you help me reset my password?"
- Agent confirms: "I see you're logged in with a temporary session."
- Agent retrieves: "Security policy requires email verification for password resets."
- Agent instructs: "Sent verification link to your registered email address."
- Customer confirms: "I clicked the link. What's next?"
- Agent prepares: "Ready to guide you through setting a new password."

Agent Examples



Business Analyst Agent

1

Role (System Prompt)

"You are a senior business analyst. Your goal is to answer business questions by querying databases and creating clear, data-driven reports."

2

Tools & Data

```
sql_database(query), file_system.write(path,  
content), access_mongo_collection()
```

Access to production sales database, marketing docs

3

Current Instruction

Generate a report on our top 3 performing products from last quarter by revenue.

4

Memory

Stores query history, previous analysis results, and learned patterns about data structure. Enables the agent to reference past insights and avoid redundant queries.

AAIS: Marketing Campaign Launch



Market Research Agent

Identifies target demographics, analyzes competitors, and uncovers market trends.



Content Creation Agent

Develops compelling copy, engaging visuals, and multimedia assets for marketing.



Campaign Execution Agent

Deploys campaigns across platforms, manages budgets, and schedules releases.



Performance Analysis Agent

Monitors campaign metrics, generates reports, and optimizes campaigns.

Execution Trace

Thought: I need to query the sales DB to find the top 3 products by revenue for last quarter.

Action: `sql_database("SELECT product_name, SUM(revenue) as total_revenue FROM sales WHERE date >= '2024-10-01' AND date <= '2024-12-31' GROUP BY product_name ORDER BY total_revenue DESC LIMIT 3")`

Observation: Results show:

1. Enterprise Software License: \$2.4M
2. Cloud Storage Pro: \$1.8M
3. Analytics Dashboard: \$1.2M

Thought: Now I'll create a formatted report with these findings.

Action: `file_system.write("Q4_2024_Top_Products_Report.md", "# Top 3 Products by Revenue - Q4 2024\n\n## Summary\n...")`

Observation: Report successfully written to Q4_2024_Top_Products_Report.md

Final Answer: I've analyzed the sales data and created a comprehensive report. The top 3 products by revenue were Enterprise Software License (\$2.4M), Cloud Storage Pro (\$1.8M), and Analytics Dashboard (\$1.2M).

The ReAct Paradigm: Think → Act → Learn:



Key Takeaways: Agent-Driven Development

The 4 Components of an Agent

- **Role:** Defines the agent's purpose and persona.
- **Engine:** LLM for reasoning.
- **Tools:** Access to external functions and data.
- **Instruction:** Guides behavior and constraints.

The ReAct Pattern (Reasoning and Acting)

- **Thought:** Agent reasons about the current state and next step.
- **Action:** Executes a specific tool or function.
- **Observation:** Receives results from the action, informing future thoughts.

Multi-Agent Communication Patterns

- **Sequential:** Tasks passed linearly from one agent to the next.
- **Parallel:** Independent tasks processed concurrently by multiple agents.
- **Hierarchical:** Master-subordinate coordination for complex workflows.
- **Peer-to-Peer:** Collaborative and distributed problem-solving.

ReAct: Think → Act → Learn: A DevOps Agent

Thought: "Identify root cause of high API latency"

Action: ``diagnose_logs(service='user_profile_api', time_range='past_hour')``

Observation: "Error rate 5%, high volume of database timeouts on ``get_user_data`` query."

Thought: "DB query seems to be the bottleneck. Need to analyze query performance"

Action: ``explain_query(query_id='get_user_data_query_hash')``

Observation: "Missing index on ``users.last_login_at`` col. Query scan count high."

Thought: "Create index to optimize query performance."

Action: ``create_index(table='users', column='last_login_at')``

Observation: "Index created successfully. Latency dropped by 80%."

How an Agent Sees a Tool

Python Function

```
def web_search(query: str) -> str:
    """Search the web for information"""
    response = requests.get(
        f"https://api.search.com/v1/search", params=
        {"q": query})
    return response.json()["results"]
```

Agent Tool Description

```
{
  "name": "web_search",
  "description": "Search the web for current
information about any topic",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "The search query"
      }
    },
    "required": ["query"]
  }
}
```